# AI-Based Tic Tac Toe Game using Minimax Algorithm

**Submitted By :**

Nityam Jain 25BCE11391
B.Tech Computer Science

---

## 1. Introduction

This project is a Python-based implementation of the classic Tic Tac Toe game. The objective of this project is to design a simple game that not only allows user interaction but also demonstrates the use of artificial intelligence in decision-making. The game can be played in both single-player and two-player modes, making it interactive and flexible.

---

## 2. Objective

The main objectives of this project are:

- To implement a playable Tic Tac Toe game using Python
- To understand game logic and decision-making
- To apply artificial intelligence concepts using algorithms
- To provide different difficulty levels for better user experience

---

## 3. Tools and Technologies Used

- Programming Language: Python
- Modules Used: random, copy
- Platform: VS Code / Terminal

---

## 4. Working of the Project

The game is played on a 3x3 grid where players take turns marking X and O. The program checks for winning conditions after every move, including rows, columns, and diagonals.

In single-player mode, the computer acts as the opponent and plays based on the selected difficulty level:

- Easy Mode: The computer selects moves randomly.
- Medium Mode: The computer tries to win or block the user's winning move.
- Hard Mode: The computer uses the Minimax algorithm to choose the best possible move.

The Minimax algorithm evaluates all possible game states and selects the move that maximizes the computer's chances of winning while minimizing the opponent's chances.

---

## 5. Features

- Single-player and two-player modes
- Multiple difficulty levels
- Intelligent AI-based gameplay
- Input validation for correct moves
- Clear display of the game board

---

## 6. Algorithm Used

The project uses the Minimax algorithm in hard mode. This algorithm works by simulating all possible moves and selecting the optimal one based on the outcome. It ensures that the computer always plays the best move, making it very difficult for the user to win.

---

## 7. Advantages

- Simple and easy to understand
- Demonstrates AI concepts clearly
- Improves logical and problem-solving skills
- Interactive and engaging gameplay

---

## 8. Limitations

- Limited to a 3x3 grid
- No graphical user interface (CLI-based)

---

## 9. Future Enhancements

- Adding a graphical user interface (GUI)
- Implementing score tracking
- Adding sound effects and animations
- Using machine learning for adaptive gameplay

---

## 10. Conclusion

This project successfully demonstrates how artificial intelligence can be applied in a simple game like Tic Tac Toe. It helped in understanding programming concepts, game logic, and algorithm-based decision-making. The implementation of the Minimax algorithm makes the game more challenging and highlights the importance of AI in problem-solving.

---

## 11. References


- Python Documentation
- Online tutorials on Tic Tac Toe and Minimax Algorithm

# Screenshots for the program and the output :

```python
import random

def print_board(board):
    print("\n")
    for i, row in enumerate(board):
        print(" | ".join(row))
        if i != 2:
            print("--+---+--")
    print("\n")

def check_winner(board, symbol):
    for i in range(3):
        if all(cell == symbol for cell in board[i]):    # Rows
            return True
        if all(board[j][i] == symbol for j in range(3)): # Columns
            return True
    # Diagonals
    if all(board[i][i] == symbol for i in range(3)):
        return True
    if all(board[i][2-i] == symbol for i in range(3)):
        return True
    return False

def is_full(board):
```

```python
def is_full(board):
    return all(cell != " " for row in board for cell in row)

def get_computer_move(board):
    empty_cells = [(i, j) for i in range(3) for j in range(3) if board[i][j] == " "]
    return random.choice(empty_cells) if empty_cells else (None, None)

def play_single_player():
    board = [[" " for _ in range(3)] for _ in range(3)]
    user_symbol = "X"
    computer_symbol = "O"
    current_turn = "user"

    print("Single Player Mode 🤖")
    print_board(board)

    while True:
        if current_turn == "user":
            try:
                row = int(input("Your turn! Enter row (0-2): "))
                col = int(input("Enter col (0-2): "))
                if row not in range(3) or col not in range(3):
                    print("Row and column must be 0, 1, or 2. Try again.")
                    continue
```

```
                    print("Row and column must be 0, 1, or 2. Try again.")
                    continue
                if board[row][col] != " ":
                    print("Cell already taken! Try again.")
                    continue
                board[row][col] = user_symbol
            except (ValueError, IndexError):
                print("Invalid input. Enter row and column as numbers between 0 and 2.
                continue
        else:
            print("Computer's turn...")
            row, col = get_computer_move(board)
            board[row][col] = computer_symbol

        print_board(board)

        if check_winner(board, user_symbol):
            print("🎉 You win!")
            return
        elif check_winner(board, computer_symbol):
            print("💻 Computer wins!")
            return
        elif is_full(board):
            print("It's a tie!")
```

```
            print("It's a tie!")
            return

        current_turn = "computer" if current_turn == "user" else "user"

def play_two_player():
    board = [[" " for _ in range(3)] for _ in range(3)]
    symbols = ["X", "O"]
    current_turn = 0  # 0 for player 1, 1 for player 2

    print("Two Player Mode 🧑♂  🧑♀  ")
    print_board(board)

    while True:
        player = f"Player {current_turn+1}"
        try:
            row = int(input(f"{player}'s turn ({symbols[current_turn]}). Enter row (0-
            col = int(input(f"{player}'s turn ({symbols[current_turn]}). Enter col (0-
            if row not in range(3) or col not in range(3):
                print("Row and column must be 0, 1, or 2. Try again.")
                continue
            if board[row][col] != " ":
                print("Cell already taken! Try again.")
                continue
```

```
                continue
            if board[row][col] != " ":
                print("Cell already taken! Try again.")
                continue
            board[row][col] = symbols[current_turn]
        except (ValueError, IndexError):
            print("Invalid input. Enter row and column as numbers between 0 and 2.")
            continue

        print_board(board)

        if check_winner(board, symbols[current_turn]):
            print(f"🎉 {player} wins!")
            return
        elif is_full(board):
            print("It's a tie!")
            return

        current_turn = 1 - current_turn  # Switch player

def main():
    print("Welcome to Tic-Tac-Toe!")
    while True:
        mode = ""
```

```python
            return

        current_turn = 1 - current_turn  # Switch player

def main():
    print("Welcome to Tic-Tac-Toe!")
    while True:
        mode = ""
        while mode not in ["1", "2"]:
            mode = input("Enter 1 to play against computer, 2 for two player mode: ")

        if mode == "1":
            play_single_player()
        else:
            play_two_player()

        play_again = input("Do you want to play again? (y/n): ").strip().lower()
        if play_again != "y":
            print("Thanks for playing!")
            break

if __name__ == "__main__":
    main()
```

```
Welcome to Tic-Tac-Toe!
Enter 1 to play against computer, 2 for two player mode: 1
Single Player Mode 🤖


  |   |
--+---+--
  |   |
--+---+--
  |   |


Your turn! Enter row (0-2): 2
Enter col (0-2): 1


  |   |
--+---+--
  |   |
--+---+--
  | X |
Invalid input. Enter row and column as numbers
Your turn! Enter row (0-2): Invalid input. Ente
0 and 2.
Your turn! Enter row (0-2): 2
Enter col (0-2): 0


  |   |
--+---+--
  | O |
--+---+--
X | X |


Computer's turn...


  |   |
--+---+--
  | O |
--+---+--
X | X | O
```

```
Your turn! Enter row (0-2): 0
Enter col (0-2): 0


X |   |
--+---+--
  | O |
--+---+--
X | X | O


Computer's turn...


X |   |
--+---+--
O | O |
--+---+--
X | X | O
```

---

```
X | O |
--+---+--
O | O | X
--+---+--
X | X | O


Your turn! Enter row (0-2): 0
Enter col (0-2): 2


X | O | X
--+---+--
O | O | X
--+---+--
X | X | O


It's a tie!
Do you want to play again? (y/n):
Thanks for playing!
```